

Exp No.9
22-09-26

Mini Project – House diction Using Machine Learning

AIM

To predict house prices using property-related features by implementing machine learning regression models and evaluating their performance using MAE, RMSE, and R^2 score.

ALGORITHM

1. **Data Collection:** Obtain the house price dataset containing property details and corresponding prices.
2. **Data Preprocessing:** Load the dataset and handle missing values and inconsistent data.
3. **Feature Selection:** Select relevant features such as area, number of bedrooms, bathrooms, balconies, floor number, facing, property type, property age, and society.
4. **Data Transformation:** Convert categorical features into numerical form using One-Hot Encoding.
5. **Train-Test Split:** Divide the dataset into training and testing sets using an 80:20 ratio.
6. **Model Training:** Train Linear Regression, Random Forest, and Gradient Boosting regression models.
7. **Model Comparison:** Compare the models based on their MAE, RMSE, and R^2 scores.
8. **Best Model Selection:** Select the model with the highest R^2 score.
9. **Prediction:** Use the selected model to predict the price of a house based on the entered property details.
10. **Web Application:** Develop a Flask-based web interface for entering property details and displaying the predicted house price.

index.html

```
index.html 1 X
HP > NEW_hpp > templates > <> index.html > html > head > style > .form-grid
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4  <meta charset="UTF-8">
5  <meta name="viewport"
6  |   content="width=device-width, initial-scale=1.0">
7  <title>HouseValue AI</title>
8  <style>
9  * {
10 |   margin: 0;
11 |   padding: 0;
12 |   box-sizing: border-box;
13 | }
14 :root {
15 |   --bg: #080a0f;
16 |   --card: #11141b;
17 |   --card-light: #181c24;
18 |   --border: rgba(255,255,255,0.09);
19 |   --text: #f5f7fa;
20 |   --muted: #8d94a1;
21 |   --accent: #ff0000;
22 |   --accent-light: #ff9090;
23 |   --green: #36d59c;
24 | }
25 body {
26 |   min-height: 100vh;
27 |   background:
28 |     radial-gradient(
29 |       circle at 10% 10%,
30 |       rgba(108,124,255,0.15),
31 |       transparent 30%
```

```
index.html 1 X
HP > NEW_hpp > templates > <> index.html > html > head > style > .navbar
2  <html lang="en">
3  <head>
4  <style>
165 | }
166 | .subtitle {
167 |   color: var(--muted);
168 |   font-size: 12px;
169 |   margin-top: 5px;
170 | }
171 | .badge {
172 |   padding: 7px 11px;
173 |   border-radius: 20px;
174 |   background:
175 |     rgba(108,124,255,0.1);
176 |   border:
177 |     1px solid
178 |     rgba(108,124,255,0.2);
179 |   color: var(--accent-light);
180 |   font-size: 10px;
181 |   font-weight: 700;
182 | }
183 | .form-grid {
184 |   display: grid;
185 |   grid-template-columns:
186 |     1fr
187 |     1fr;
188 |   gap: 19px;
189 | }
190 | .form-group {
191 |   display: flex;
192 | }
```

App.py

```
index.html | app.py X
HP > NEW_hpp > app.py > ...
1 from flask import Flask
2 from flask import render_template
3 Click to add a breakpoint >rt request
4 import pandas as pd
5 import joblib
6
7
8 app = Flask(__name__)
9 model = joblib.load(
10     "house_price_model.pkl"
11 )
12 model_info = joblib.load(
13     "model_info.pkl"
14 )
15 MODEL_NAME = model_info[
16     "model_name"
17 ]
18 MODEL_R2 = model_info[
19     "r2"
20 ]
21 MODEL_MAE = model_info[
22     "mae"
23 ]
24 MODEL_RMSE = model_info[
25     "rmse"
26 ]
27 @app.route("/")
28 def home():
29     return render_template(
30         "index.html",
```

```
index.html | app.py X
HP > NEW_hpp > app.py > ...
42 def predict():
43     bedrooms = int(
44         request.form["bedrooms"]
45     )
46     bathrooms = int(
47         request.form["bathrooms"]
48     )
49     balcony = float(
50         request.form["balcony"]
51     )
52     floor = float(
53         request.form["floor"]
54     )
55     facing = request.form[
56         "facing"
57     ]
58     age_group = request.form[
59         "age_group"
60     ]
61     property_type = request.form[
62         "property_type"
63     ]
64     society = request.form[
65         "society"
66     ]
67     input_data = pd.DataFrame({
68         "area": [area],
69         "bedRoom": [bedrooms],
70         "bathRoom": [bathrooms],
71         "balcony": [balcony],
72         "floorNum": [floor],
73         "facing": [facing],
74         "age_group": [age_group],
75         "property_type": [property_type],
76         "society": [society]
77     })
78     prediction = model.predict(
79         input_data
```

train_model.py

```
> index.html | app.py | train_model.py X
HP > NEW_hpp > train_model.py > ...
1  import pandas as pd
2  import joblib
3
4  from sklearn.model_selection import train_test_split
5  from sklearn.preprocessing import OneHotEncoder
6  from sklearn.compose import ColumnTransformer
7  from sklearn.pipeline import Pipeline
8  from sklearn.impute import SimpleImputer
9
10 from sklearn.linear_model import LinearRegression
11 from sklearn.ensemble import RandomForestRegressor
12 from sklearn.ensemble import GradientBoostingRegressor
13
14 from sklearn.metrics import mean_absolute_error
15 from sklearn.metrics import mean_squared_error
16 from sklearn.metrics import r2_score
17
18
19
20 data = pd.read_csv("house_cleaned.csv")
21
22 print("=====")
23 print("      HOUSE PRICE PREDICTION")
24 print("=====")
25
26 print("\nOriginal dataset shape:", data.shape)
27
28
29
30 data = data.dropna(subset=["price"])
31
32 print(
33     "Dataset after removing missing prices:",
34     data.shape
35 )
36
37
38 def clean_balcony(value):
39
```

```
> index.html | app.py | train_model.py X
HP > NEW_hpp > train_model.py > ...
288 }
289 results = {}
290 trained_models = {}
291 print("\n")
292 print("=====")
293 print("      MODEL COMPARISON")
294 print("=====")
295 for name, model in models.items():
296     print(
297         f"\nTraining {name}..."
298     )
299     pipeline = Pipeline(
300         steps=[
301             (
302                 "preprocessor",
303                 preprocessor
304             ),
305             (
306                 "model",
307                 model
308             )
309         ]
310     )
311     pipeline.fit(
312         X_train,
313         y_train
314     )
315     predictions = pipeline.predict(
316         X_test
317     )
318     mae = mean_absolute_error(
319         y_test,
320         predictions
321     )
322     mse = mean_squared_error(
323         y_test,
324         predictions
325     )
```

Result:

The House Price Prediction web application was successfully developed and implemented. The system allows users to enter property details such as area, number of bedrooms, bathrooms, balconies, floor number, facing, property type, property age, and society, and obtain a predicted house price.